

HW8

Execution description

Q1:

<https://colab.research.google.com/drive/1XGiBLHyjKZX3TKMIksDgVq64wMFvPwfp?usp=sharing>

Q2:

<https://colab.research.google.com/drive/1z7ZLQiK2YNQF9UVNmmYsdSeMnBysEwdR?usp=sharing>

Experimental results

Q1: 更動部分

```
class CompanyStockData(Dataset):
    def __init__(self, dataframe, window=10):
        super().__init__()
        self.df = dataframe
        self.window = window

    def __len__(self):
        return len(self.df) - self.window

    def __getitem__(self, idx):
        window_s = idx
        window_t = idx + self.window
        feature = self.df.iloc[window_s:window_t - 1]
        feature = feature[['open', 'close', 'low', 'high']].values
        feature = torch.from_numpy(feature).float()
        target = self.df.iloc[window_t - 1]
        target = target['volume'].item()
        target = torch.tensor([target]).float()
        return feature, target

WINDOW = 16
train_set = CompanyStockData(df_train, window=WINDOW)
valid_set = CompanyStockData(df_valid, window=WINDOW)
print('#train:', len(train_set))
print('#valid:', len(valid_set))

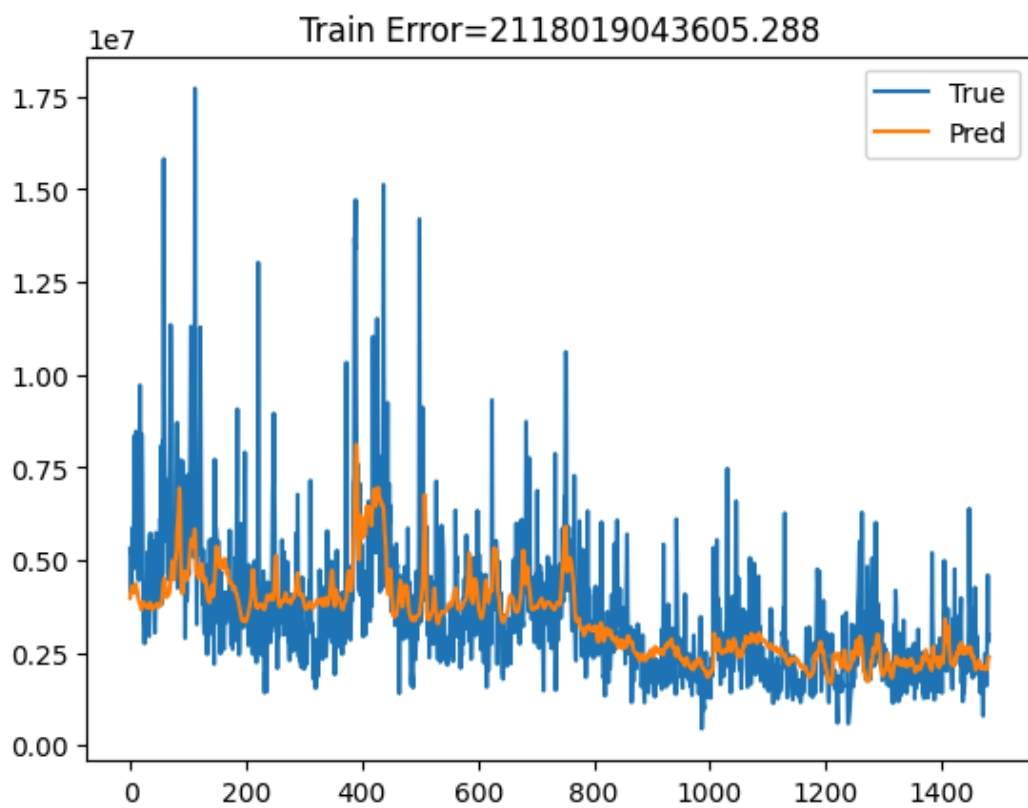
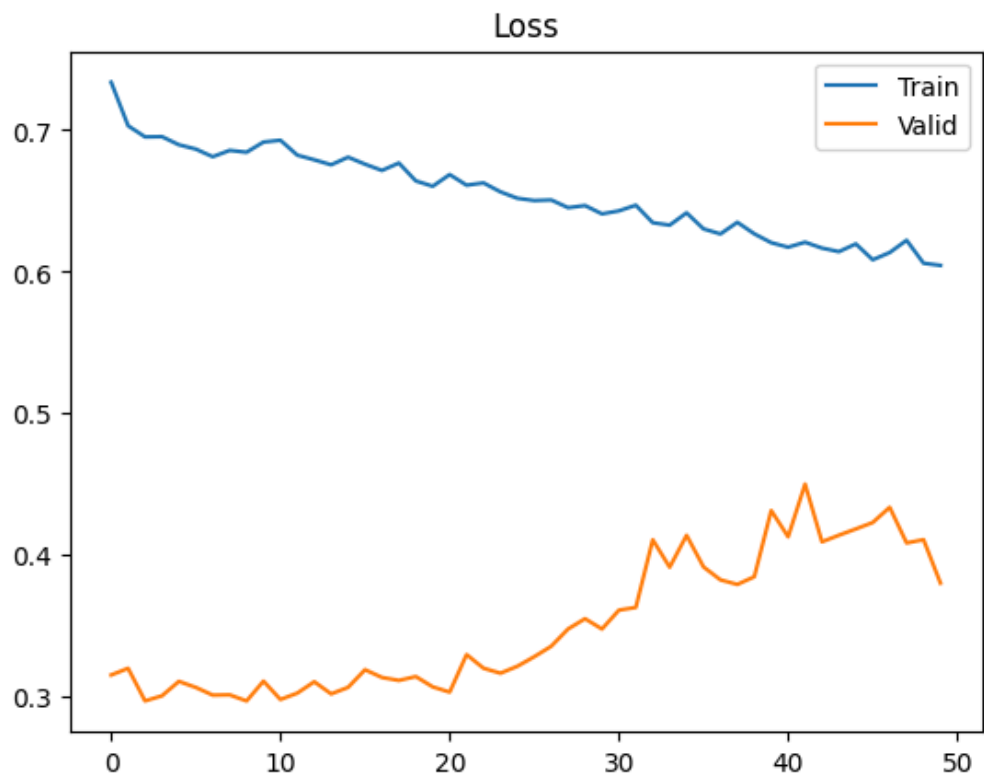
feature, target = valid_set[100]
print(feature)
print(target)
```

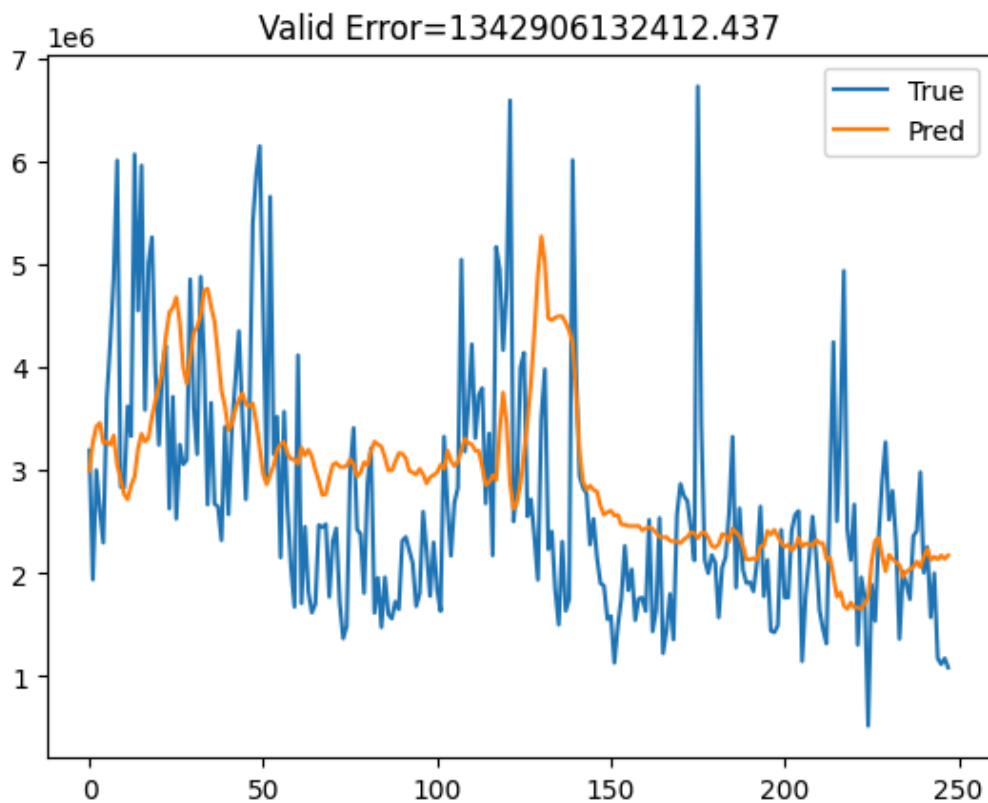
```
@torch.no_grad()
def evaluate(dataset):
    model.eval()
    preds = []
    trues = []
    loader = DataLoader(dataset, batch_size=32, shuffle=False, num_workers=4)
    for feature_b, target_b in tqdm(iter(loader)):
        predict_b = model(feature_b.to(device)).to('cpu')
        preds.extend(predict_b)
        trues.extend(target_b)
    preds = torch.stack(preds).numpy() # [N]
    trues = torch.stack(trues).numpy() # [N]
    return preds, trues

preds, trues = evaluate(train_set)
preds = preds * std.scale_[4] + std.mean_[4]
trues = trues * std.scale_[4] + std.mean_[4]
error = ((preds - trues) ** 2).mean()
fig, ax = plt.subplots()
ax.plot(trues, label='True')
ax.plot(preds, label='Pred')
ax.set_title(f'Train Error={error:.3f}')
ax.legend()
plt.show()

preds, trues = evaluate(valid_set)
preds = preds * std.scale_[4] + std.mean_[4]
trues = trues * std.scale_[4] + std.mean_[4]
error = ((preds - trues) ** 2).mean()
fig, ax = plt.subplots()
ax.plot(trues, label='True')
ax.plot(preds, label='Pred')
ax.set_title(f'Valid Error={error:.3f}')
ax.legend()
plt.show()
```

需把 dataset 那邊的 high 改成 volume, 以及將下方 **Evaluation** 的部分裡的 index 改為 4 (feature 裡面 volume 在第五個)





Q2: 更動部分

```
df_stt = pd.read_csv(csv_path_stt) # a DataFrame object
df_stt = df_stt.sort_values(by='date') # csv are not guaranteed to be ordered by date
df_stt.head(10)

df_all = pd.read_csv(csv_path_all)
df_ual = df_all[df_all['symbol'] == 'UAL']
df_ual = df_ual.sort_values(by='date')
df_ual = df_ual.reset_index(drop=True)
```

```
pivot = round(0.85 * len(df_ual))
print('pivot:', df_ual.loc[pivot, 'date'])

df_train = df_ual[:pivot].copy() # DataFrame
df_valid = df_ual[pivot:].copy() # DataFrame

print('pivot:', df_stt.loc[pivot, 'date'])

df_train2 = df_stt[:pivot].copy() # DataFrame
df_valid2 = df_stt[pivot:].copy() # DataFrame

FEATURES = ['open', 'close', 'low', 'high', 'volume']
TARGET = ['close']
std1 = StandardScaler().fit(df_train[FEATURES])
std2 = StandardScaler().fit(df_train2[TARGET])
df_train[FEATURES] = std1.transform(df_train[FEATURES])
df_valid[FEATURES] = std1.transform(df_valid[FEATURES])

df_train.head(10)

df_train2[TARGET] = std2.transform(df_train2[TARGET])
df_valid2[TARGET] = std2.transform(df_valid2[TARGET])

df_train2.head(10)
```

資料集切割的部分 ual, stt 都要先準備好

```

class CompanyStockData(Dataset):
    def __init__(self, dataframe, dataframe2, window=10):
        super().__init__()
        self.df = dataframe
        self.df2 = dataframe2
        self.window = window

    def __len__(self):
        return len(self.df) - self.window

    def __getitem__(self, idx):
        window_s = idx
        window_t = idx + self.window
        feature = self.df.iloc[window_s:window_t - 1]
        feature = feature[['open', 'high', 'low', 'volume']].values
        feature = torch.from_numpy(feature).float()
        target = self.df2.iloc[window_t - 1]
        target = target['close'].item()
        target = torch.tensor([target]).float()
        return feature, target

```

dataset 這裡, 兩個公司的 input 都要做好, 因為要用 ual 的前 15 天去預測 stt 第 16 天

```

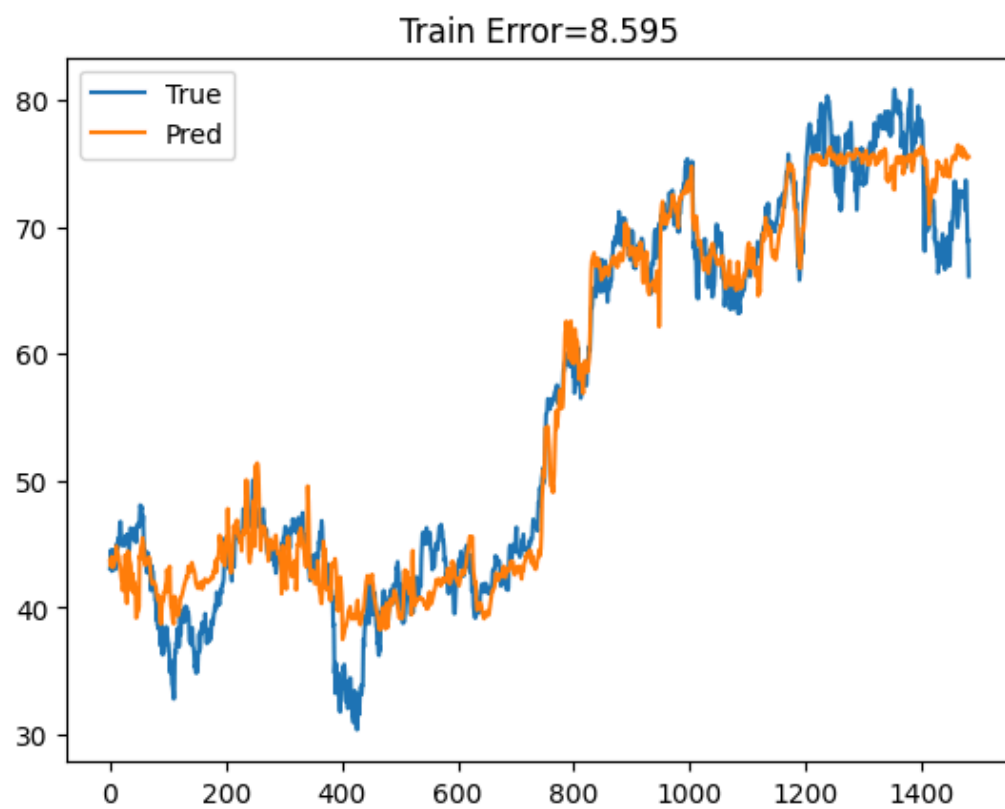
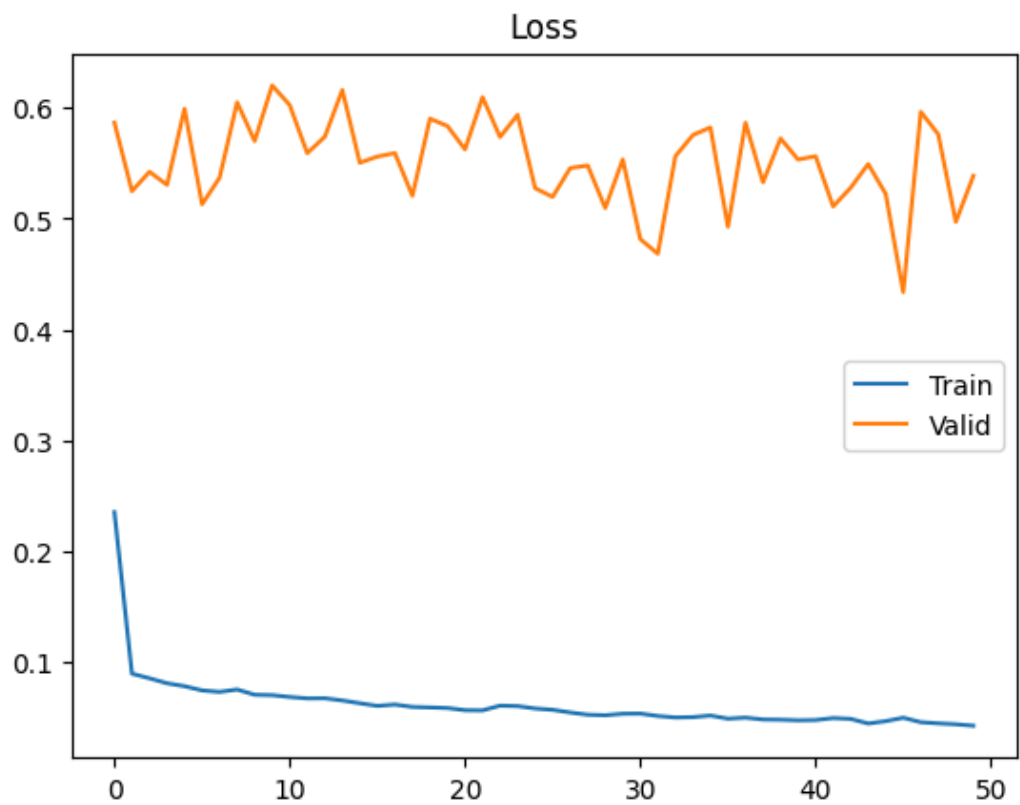
@torch.no_grad()
def evaluate(dataset):
    model.eval()
    preds = []
    trues = []
    loader = DataLoader(dataset, batch_size=32, shuffle=False, num_workers=4)
    for feature_b, target_b in tqdm(iter(loader)):
        predict_b = model(feature_b.to(device)).to('cpu')
        preds.extend(predict_b)
        trues.extend(target_b)
    preds = torch.stack(preds).numpy() # [N]
    trues = torch.stack(trues).numpy() # [N]
    return preds, trues

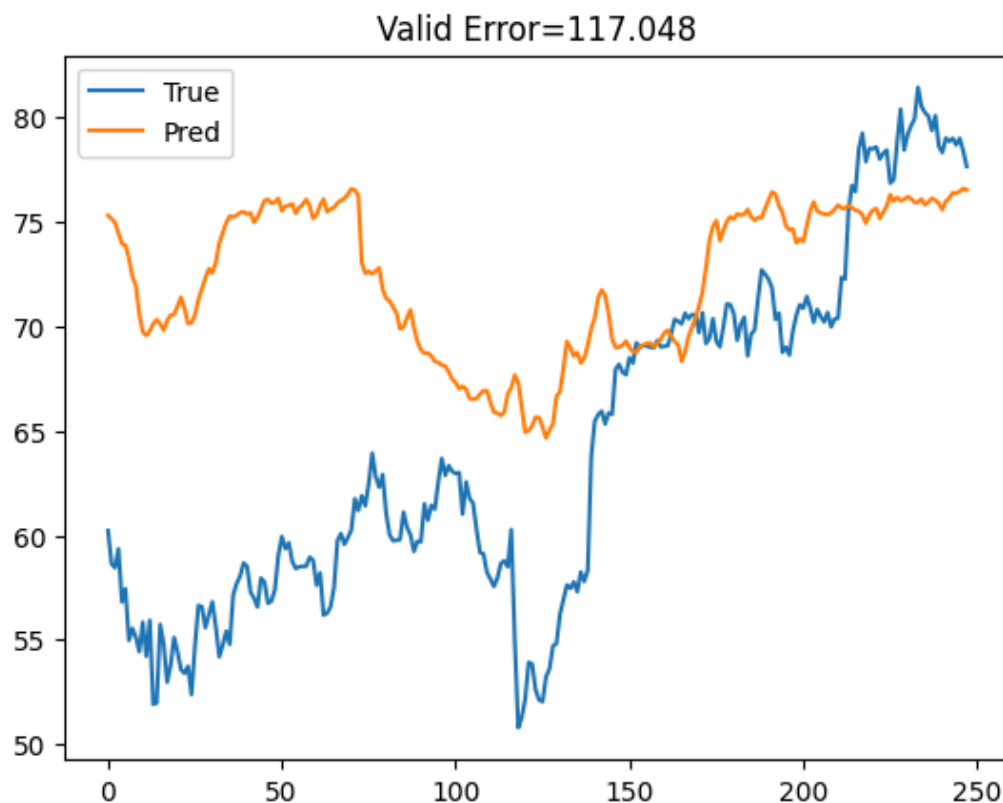
preds, trues = evaluate(train_set)
preds = preds * std2.scale_[0] + std2.mean_[0]
trues = trues * std2.scale_[0] + std2.mean_[0]
error = ((preds - trues) ** 2).mean()
fig, ax = plt.subplots()
ax.plot(trues, label='True')
ax.plot(preds, label='Pred')
ax.set_title(f'Train Error={error:.3f}')
ax.legend()
plt.show()

preds, trues = evaluate(valid_set)
preds = preds * std2.scale_[0] + std2.mean_[0]
trues = trues * std2.scale_[0] + std2.mean_[0]
error = ((preds - trues) ** 2).mean()
fig, ax = plt.subplots()
ax.plot(trues, label='True')
ax.plot(preds, label='Pred')
ax.set_title(f'Valid Error={error:.3f}')
ax.legend()
plt.show()

```

Evaluation 部分要改用 ual 的 StandardScaler





Conclusion

本次作業實作了兩個基於 LSTM 的時間序列預測任務。

在**第一題**中，我們將範例程式從預測 STT 的最高價改為預測成交量（volume）。

在**第二題**中，則使用 UAL 前 15 天的股價資料來預測 STT 第 16 天的收盤價。

兩題皆使用 MSE 作為評估指標，並繪製預測與真實值的比較圖。

Discussion

第一題 中，由於成交量波動較大且受市場行為影響，預測難度比價格高，模型表現也較不穩定。

第二題 屬於跨公司預測，測試 UAL 的股價走勢是否能協助推估 STT 的未來收盤價。結果顯示兩公司之間的關聯性對預測效果有顯著影響。

在兩題中，驗證集的表現能有效觀察模型是否過擬合，並評估其在未知資料上的泛化能力。

